Project Report: Web-Based Text Encryptor/Decryptor

Team Members Details :

Name : Utkarsh Singh

Roll no : 2401010099

Btech Cse Core

Section-C

Name :Aayush

Roll no : 2401010042

Btech Cse Core

Section-C

Name :Manan Sehgal

Roll no : 2401010043

Btech Cse Core

Section-C

Acknowledgement

We express our profound gratitude to K.R. Mangalam University for providing us with the opportunity to undertake this project. We sincerely thank our professors and mentors for their continuous encouragement, guidance, and inspiration. This project would not have been possible without the collaborative spirit and endless support from our peers, friends, and family members who motivated us at every step.We are particularly grateful to our project mentor whose critical feedback and valuable suggestions helped us refine and complete this project. We, Utkarsh Singh , Aayush and Manan Sehgal, extend heartfelt thanks to everyone who contributed directly or indirectly to this academic endeavor.

1. Introduction

In the digital age, information security is paramount. This project focuses on building a simple yet effective web-based **Text Encryptor/Decryptor** that allows users to encode and decode messages securely. Developed using HTML, CSS, and JavaScript, the application serves as an educational tool to demonstrate client-side encryption techniques while offering an intuitive user interface.

The main objective is to provide a minimal, interactive environment where users can input text, encrypt or decrypt it, and view their history of operations. This lightweight tool is especially useful for students, developers, or anyone interested in understanding the basics of encoding and decoding text in a web environment.

---

2. Project Objectives

The project was developed with the following goals in mind:

- Create a user-friendly interface for text encryption and decryption.
- Implement secure base64 encoding and decoding using JavaScript.
- Maintain a real-time log of operations for user reference.
- Use modern responsive design practices for optimal viewing across devices.

3. System Design and Architecture

Frontend Components

The entire application runs on the frontend using native web technologies:

- **HTML** defines the structure and layout of the application.
- **CSS** provides theming, styling, and responsiveness using modern practices like CSS variables and flexbox.
- **JavaScript** handles the core functionality including encryption, decryption, and history management.

Functional Workflow

1. **Input Section**: Users enter plaintext or encoded text in the first text area.
2. **Action Buttons**: Users choose to either encrypt or decrypt the content using corresponding buttons.
3. **Output Section**: The processed text appears in a read-only field for copying or reference.
4. **History Logging**: Each action (encryption or decryption) is recorded and displayed in a scrollable history log.

---

4. User Interface Description

The UI design follows a clean and dark theme that enhances readability and reduces eye strain. Key interface components include:

- **Container**: The main card that wraps all UI elements with padding and a drop shadow for depth.
- **Text Areas**: Allow for multiline input and output, styled with a modern dark background and light text.
- **Buttons**: Clearly labeled with “Encrypt” and “Decrypt,” styled with hover effects for better interactivity.
- **History Section**: A collapsible list view that dynamically appends records of past encryption/decryption attempts.

CSS variables (e.g., --bg-color, --accent-color) enable easy customization of the color scheme, making the interface theme-consistent and adaptable.

---

5. Core Features

Encryption Functionality

Implemented using the btoa() function, the encryption process converts the input string to a base64-encoded version. This simulates a form of obfuscation that hides the original text.

javascript

CopyEdit

const encrypted = btoa(unescape(encodeURIComponent(input)));

The function ensures that special Unicode characters are properly encoded before base64 conversion.

Decryption Functionality

The atob() function decodes a base64 string back to its original text. Proper decoding is ensured by first using escape() and decodeURIComponent() for character handling.

javascript

CopyEdit

const decrypted = decodeURIComponent(escape(atob(input)));

An error-handling mechanism using try-catch ensures that malformed or invalid inputs result in a user-friendly error message.

History Log

Each encryption or decryption event is logged with a timestamp-style label (e.g., “Encrypted: [text]”) and appended to the top of the history list. This enhances usability and provides a running audit of operations.

javascript

CopyEdit

```
function updateHistory(entry) {
  const item = document.createElement('li');
  item.textContent = entry;
  list.insertBefore(item, list.firstChild);
}
```

---

6. Security Considerations

While this project uses base64 encoding, it's important to understand that **base64 is not encryption** in the cryptographic sense. It merely encodes data into a readable ASCII format. This tool does **not offer real security** and should not be used for sensitive data.

For actual cryptographic protection, methods such as AES or RSA would be needed, ideally handled on the backend or using libraries like CryptoJS on the client side.

---

7. Limitations

- **Security**: The use of base64 encoding provides no real cryptographic security.
- **Client-side Storage**: The history is session-based and not persistent after a page reload.
- **Scalability**: This tool is best suited for short strings; it is not optimized for large-scale data processing.
- **No Backend**: There’s no data persistence, server-side validation, or user authentication.

---

8. Future Enhancements

Some ideas to enhance the project in the future include:

- **Add Persistent History**: Use localStorage or IndexedDB to preserve logs across sessions.
- **Stronger Encryption**: Integrate secure encryption algorithms like AES using libraries such as crypto.subtle or CryptoJS.
- **Copy-to-Clipboard Feature**: Enable users to quickly copy output text.
- **Theme Switcher**: Allow users to toggle between dark and light modes.
- **Mobile Optimization**: Improve layout responsiveness for small screens.

---

9. Conclusion

This project demonstrates how a basic text encryptor/decryptor can be created using only frontend technologies. It achieves its educational purpose by showing the transformation between plain and encoded text, all within a visually appealing and interactive interface.

Although simple, the project can be a starting point for more advanced applications involving secure messaging, data encoding, or cryptographic

learning tools. It highlights the power of modern web development for building useful utilities with minimal resources.